

13 — Multi-session encoding, decoding, and null controls

This version is multi-session by construction. It analyzes every session with the required feature, tensor, and embedding assets, and writes all outputs into a versioned publication namespace.

Setup and multi-session asset index

Run this first. The asset index tells you exactly which sessions are ready for latent decoding, population encoding, full neural analysis, and cross-session follow-up.

```
from pathlib import Path
import os
import sys
import yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Edit this path only if your project has moved.
PROJECT_ROOT = Path(
    os.environ.get(
        "LATENT_MANIFOLD_PROJECT_ROOT",
        r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural"
    )
).resolve()

SRC_DIR = PROJECT_ROOT / "src"
if not PROJECT_ROOT.exists():
    raise FileNotFoundError(f"PROJECT_ROOT does not exist: {PROJECT_ROOT}")
if not SRC_DIR.exists():
    raise FileNotFoundError(f"src directory does not exist: {SRC_DIR}")
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
os.chdir(PROJECT_ROOT)

cfg_path = PROJECT_ROOT / "configs" / "publication_upgrade.yaml"
if not cfg_path.exists():
    raise FileNotFoundError(f"Could not find publication config: {cfg_path}")
with cfg_path.open("r", encoding="utf-8") as f:
    pub_cfg = yaml.safe_load(f)
```

```

pub_cfg = yaml.safe_load(cfg)
cfg = pub_cfg

# Versioned namespace. This prevents accidental overwriting of older publications.
PUBLICATION_RUN_LABEL = os.environ.get("PUBLICATION_RUN_LABEL", "publication_upgrade")
ALLOW_CANONICAL_PUBLICATION_WRITE = os.environ.get("ALLOW_CANONICAL_PUBLICATION_WRITE", "True")
if PUBLICATION_RUN_LABEL == "publication_upgrade" and not ALLOW_CANONICAL_PUBLICATION_WRITE:
    raise RuntimeError(
        "Refusing to write into the canonical publication_upgrade namespace. "
        "Use a versioned PUBLICATION_RUN_LABEL or set ALLOW_CANONICAL_PUBLICATION_WRITE to True."
    )

class ProjectPaths:
    def __init__(self, root: Path, run_label: str):
        self.root = root
        self.configs_dir = root / "configs"
        self.data_dir = root / "data"
        self.external_dir = root / "data" / "external"
        self.raw_dir = root / "data" / "raw"
        self.interim_dir = root / "data" / "interim"
        self.processed_dir = root / "data" / "processed"
        self.versioned_processed_dir = self.processed_dir / run_label
        self.reports_dir = root / "reports"
        self.tables_dir = root / "reports" / "tables"
        self.figures_dir = root / "reports" / "figures"
        self.html_dir = root / "reports" / "html"
        self.publication_tables_dir = root / "reports" / "tables" / run_label
        self.publication_figures_dir = root / "reports" / "figures" / run_label
        self.manuscript_dir = root / "manuscript" / "top_journal_scaffold"
        for d in [
            self.versioned_processed_dir,
            self.publication_tables_dir,
            self.publication_figures_dir,
            self.manuscript_dir,
        ]:
            d.mkdir(parents=True, exist_ok=True)

paths = ProjectPaths(PROJECT_ROOT, PUBLICATION_RUN_LABEL)

def save_table(df, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
    return path

def save_figure(fig, path, dpi=300):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, dpi=dpi, bbox_inches="tight")
    return path

```

```

# Upgrade helper functions shipped with this patch.
from v1_manifold_publication.multisession_core import (
    build_multisession_asset_index,
    candidate_feature_files,
    choose_best_feature_table,
    ready_sessions,
    movie_feature_targets,
    empirical_null_summary,
    safe_read_csv,
    safe_table_index,
    claim_gate_from_assets,
    ensure_table,
)

metadata_path = paths.external_dir / "allen_v1_natural_movie_experiments.csv"
asset_index = build_multisession_asset_index(paths, metadata_path=metadata_path)
save_table(asset_index, paths.publication_tables_dir / "00_multisession_asset_index.csv")

print("Using PROJECT_ROOT:", PROJECT_ROOT)
print("Publication run label:", PUBLICATION_RUN_LABEL)
print("Publication tables:", paths.publication_tables_dir)
print("Publication figures:", paths.publication_figures_dir)
print("Versioned processed dir:", paths.versioned_processed_dir)
display(asset_index)

```

```

Using PROJECT_ROOT: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication run label: publication_upgrade_v3_multisession
Publication tables: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication figures: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Versioned processed dir: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold

```

	session_id	feature_file	feature_shape	
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500855614.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500855614.csv
1	500964514	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500964514.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500964514.csv
2	501271265	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\501271265.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\501271265.csv

```

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movie-experiments"
).resolve()

sessions = ["500855614", "500964514", "501271265"]

```

```

rows = []

for session_id in sessions:
    feature_files = sorted(
        (PROJECT_ROOT / "data" / "processed").glob(
            f"**/session_{session_id}_publication_enhanced_frame_features.cs
        )
    )

    tensor_file = (
        PROJECT_ROOT
        / "data"
        / "interim"
        / f"session_{session_id}_natural_movie_one_tensor.h5"
    )

    embedding_file = (
        PROJECT_ROOT
        / "data"
        / "processed"
        / f"session_{session_id}_embeddings.npz"
    )

    rows.append({
        "session_id": session_id,
        "n_feature_files": len(feature_files),
        "has_tensor": tensor_file.exists(),
        "tensor_file": str(tensor_file),
        "has_embeddings": embedding_file.exists(),
        "embedding_file": str(embedding_file),
        "ready_for_notebook_13": bool(
            feature_files
            and tensor_file.exists()
            and embedding_file.exists()
        ),
    })

readiness_check = pd.DataFrame(rows)
display(readiness_check)

```

	session_id	n_feature_files	has_tensor	tensor_file
0	500855614	2	True	\Users\Peter\Documents\projects\NeuroA
1	500964514	2	True	\Users\Peter\Documents\projects\NeuroA

```

# -----
# Generate missing embedding files from existing tensor files
..

```

```

# -----
# This creates session_{id}_embeddings.npz for sessions that already have
# tensors but no embedding file.
# -----

from sklearn.decomposition import PCA
from sklearn.manifold import Isomap

try:
    from umap import UMAP
    UMAP_AVAILABLE = True
except Exception as exc:
    UMAP_AVAILABLE = False
    print("UMAP is not available. UMAP embedding will be skipped.")
    print("Reason:", repr(exc))

from v1_manifold.preprocessing import load_trial_tensor_h5
from v1_manifold.features import standardize_matrix

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
").resolve()

interim_dir = PROJECT_ROOT / "data" / "interim"
processed_dir = PROJECT_ROOT / "data" / "processed"

missing_embedding_sessions = ["500964514", "501271265"]

random_seed = int(cfg.get("project", {}).get("random_seed", 42))

rows = []

for session_id in missing_embedding_sessions:
    tensor_candidates = sorted(
        interim_dir.glob(f"session_{session_id}*_tensor.h5")
    )

    if not tensor_candidates:
        rows.append({
            "session_id": session_id,
            "status": "failed",
            "reason": "No tensor file found.",
        })
        continue

    tensor_path = tensor_candidates[0]
    emb_path = processed_dir / f"session_{session_id}_embeddings.npz"

    print("\n" + "=" * 80)
    print("Session:", session_id)
    print("Tensor:", tensor_path)

```

```

print("Output embedding file:", emb_path)

try:
    tensor, R = load_trial_tensor_h5(tensor_path)

    print("Tensor shape:", tensor.shape)
    print("Population matrix R shape:", R.shape)

    Xz, scaler = standardize_matrix(R)

    print("Standardized matrix shape:", Xz.shape)

    n_samples, n_cells = Xz.shape

    embedding_dict = {
        "frame": np.arange(n_samples),
    }

    # -----
    # PCA, 3D
    # -----
    pca_dim = min(3, n_samples, n_cells)
    pca = PCA(n_components=pca_dim, random_state=random_seed)
    pca_z = pca.fit_transform(Xz)

    if pca_z.shape[1] < 3:
        pca_z = np.pad(
            pca_z,
            ((0, 0), (0, 3 - pca_z.shape[1])),
            mode="constant",
        )

    embedding_dict["pca"] = pca_z

    # -----
    # PCA full, up to 20D
    # -----
    pca_full_dim = min(20, n_samples, n_cells)
    pca_full = PCA(n_components=pca_full_dim, random_state=random_seed)
    embedding_dict["pca_full"] = pca_full.fit_transform(Xz)

    # -----
    # UMAP, 3D
    # -----
    if UMAP_AVAILABLE:
        try:
            umap_model = UMAP(
                n_components=3,
                n_neighbors=30,
                min_dist=0.1,
                . . . . .

```

```

        metric="euclidean",
        random_state=random_seed,
    )
    embedding_dict["umap"] = umap_model.fit_transform(Xz)
except Exception as exc:
    print("UMAP failed for session", session_id, repr(exc))

# -----
# ISOMAP, 3D
# -----
try:
    n_neighbors = min(30, max(5, n_samples - 1))
    isomap = Isomap(
        n_components=3,
        n_neighbors=n_neighbors,
    )
    embedding_dict["isomap"] = isomap.fit_transform(Xz)
except Exception as exc:
    print("ISOMAP failed for session", session_id, repr(exc))

# Save embedding file.
emb_path.parent.mkdir(parents=True, exist_ok=True)
np.savez_compressed(emb_path, **embedding_dict)

rows.append({
    "session_id": session_id,
    "status": "saved",
    "tensor_file": str(tensor_path),
    "embedding_file": str(emb_path),
    "embedding_names": ", ".join(embedding_dict.keys()),
    "n_frames": n_samples,
    "n_cells": n_cells,
})

print("Saved:", emb_path)
print("Embeddings:", {k: v.shape for k, v in embedding_dict.items()})

except Exception as exc:
    rows.append({
        "session_id": session_id,
        "status": "failed",
        "tensor_file": str(tensor_path),
        "embedding_file": str(emb_path),
        "reason": repr(exc),
    })

embedding_generation_status = pd.DataFrame(rows)

save_table(
    embedding_generation_status,
    paths.publication_tables_dir / "12c_missing_embedding_generation_status."

```

)

```
display(embedding_generation_status)
```

```
=====
Session: 500964514
Tensor: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-
Output embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manif
Tensor shape: (10, 214, 900)
Population matrix R shape: (900, 214)
Standardized matrix shape: (900, 214)
c:\Users\Peter\.neuro\Lib\site-packages\umap\umap_.py:1952: UserWarning: n_jo
  warn(
Saved: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
Embeddings: {'frame': (900,), 'pca': (900, 3), 'pca_full': (900, 20), 'umap':

=====
Session: 501271265
Tensor: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-
Output embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manif
Tensor shape: (10, 215, 900)
Population matrix R shape: (900, 215)
Standardized matrix shape: (900, 215)
c:\Users\Peter\.neuro\Lib\site-packages\umap\umap_.py:1952: UserWarning: n_jo
  warn(
Saved: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
Embeddings: {'frame': (900,), 'pca': (900, 3), 'pca_full': (900, 20), 'umap':

  session_id  status  tensor_file
0  500964514  saved  C:
  \Users\Peter\Documents\projects\NeuroAI\late...  \Users\Peter\Do
1  501271265  saved  C:
```

```
# -----
# Confirm all sessions are ready for multi-session notebook 13
# -----

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-m
").resolve()

sessions = ["500855614", "500964514", "501271265"]

rows = []

for session_id in sessions:
    feature_files = sorted(
        (PROJECT_ROOT / "data" / "processed").glob(
            f"**/session_{session_id}_publication_enhanced_frame_features.cs
        )
    )
```



```

)

tensor_file = (
    PROJECT_ROOT
    / "data"
    / "interim"
    / f"session_{session_id}_natural_movie_one_tensor.h5"
)

embedding_file = (
    PROJECT_ROOT
    / "data"
    / "processed"
    / f"session_{session_id}_embeddings.npz"
)

embedding_names = []
if embedding_file.exists():
    with np.load(embedding_file, allow_pickle=False) as emb:
        embedding_names = list(emb.files)

rows.append({
    "session_id": session_id,
    "n_feature_files": len(feature_files),
    "has_tensor": tensor_file.exists(),
    "has_embeddings": embedding_file.exists(),
    "embedding_names": ",".join(embedding_names),
    "ready_for_notebook_13": bool(
        feature_files
        and tensor_file.exists()
        and embedding_file.exists()
    ),
})

readiness_check = pd.DataFrame(rows)
display(readiness_check)

```

	session_id	n_feature_files	has_tensor	has_embeddings	
0	500855614	2	True	True	pca,pca_full,umap,iso
1	500964514	2	True	True	fra
2	501271265	2	True	True	fra

```

# -----
# Refresh multi-session asset index after generating new embeddings
# -----

```

```

PROJECT_ROOT = Path(
    r"C:\Users\Beton\Documents\projects\NeuroAT\latent_manifest_v1_natural_m

```

```

    ).resolve()

processed_dir = PROJECT_ROOT / "data" / "processed"
interim_dir = PROJECT_ROOT / "data" / "interim"

sessions = ["500855614", "500964514", "501271265"]

def choose_best_feature_file(session_id):
    candidates = sorted(
        processed_dir.glob(f"**/session_{session_id}_publication_enhanced_fr
    )

    # Exclude backup files.
    candidates = [
        p for p in candidates
        if "backup" not in p.name.lower()
    ]

    if not candidates:
        candidates = sorted(
            processed_dir.glob(f"**/session_{session_id}*frame_features.csv"
        )

    if not candidates:
        return None

    scored = []

    for path in candidates:
        try:
            head = pd.read_csv(path, nrows=5)
            scored.append({
                "path": path,
                "n_columns": len(head.columns),
                "has_movie_frame": "movie_frame" in head.columns,
                "modified_time": path.stat().st_mtime,
            })
        except Exception:
            pass

    if not scored:
        return candidates[0]

    scored = pd.DataFrame(scored).sort_values(
        ["has_movie_frame", "n_columns", "modified_time"],
        ascending=[False, False, False],
    )

    return Path(scored.iloc[0]["path"])

```

```

rows = []

for session_id in sessions:
    feature_file = choose_best_feature_file(session_id)

    tensor_candidates = sorted(
        interim_dir.glob(f"session_{session_id}*_tensor.h5")
    )
    tensor_file = tensor_candidates[0] if tensor_candidates else None

    embedding_file = processed_dir / f"session_{session_id}_embeddings.npz"

    embedding_names = []
    embedding_shapes = []

    if embedding_file.exists():
        with np.load(embedding_file, allow_pickle=False) as emb:
            for name in emb.files:
                arr = emb[name]
                embedding_names.append(name)
                embedding_shapes.append(f"{name}:{arr.shape}")

    rows.append({
        "session_id": session_id,
        "feature_file": str(feature_file) if feature_file is not None else None,
        "tensor_file": str(tensor_file) if tensor_file is not None else None,
        "embedding_file": str(embedding_file) if embedding_file.exists() else None,
        "embedding_names": ",".join(embedding_names),
        "embedding_shapes": ";" + ".join(embedding_shapes),
        "has_features": feature_file is not None,
        "has_tensor": tensor_file is not None,
        "has_embeddings": embedding_file.exists(),
        "ready_for_latent_decoding": bool(feature_file is not None and embedding_file.exists()),
        "ready_for_population_encoding": bool(feature_file is not None and tensor_file is not None),
        "ready_for_full_neural_analysis": bool(feature_file is not None and tensor_file is not None and embedding_file.exists())
    })

asset_index = pd.DataFrame(rows)

save_table(
    asset_index,
    paths.publication_tables_dir / "13_refreshed_multisession_asset_index.csv"
)

display(asset_index)

ready = asset_index.loc[
    asset_index["ready_for_latent_decoding"],
    "session_id",
]
return(str)\ to list(\

```

```
j.astype(str).tolist()
```

```
print("Sessions ready for latent decoding:", ready)
```

	session_id	feature_file
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\late... \Users\Peter\Documents\p
1	500964514	C:\Users\Peter\Documents\projects\NeuroAI\late... \Users\Peter\Documents\p
2	501271265	C:\Users\Peter\Documents\projects\NeuroAI\late... \Users\Peter\Documents\p

Sessions ready for latent decoding: ['500855614', '500964514', '501271265']

✓ Latent representation → real movie-feature decoding

This uses all sessions with saved frame-level embeddings and enhanced movie-frame features.

```
from v1_manifold_publication.decoding import evaluate_regression_cv, movie_b
from v1_manifold_publication.metrics import summarize_folds

continuous_targets = [
    "rms_contrast",
    "spatial_frequency_centroid",
    "orientation_selectivity",
    "luminance_std",
    "total_spectral_power",
    "temporal_absdiff_rms",
    "scene_change_score",
]

n_movie_blocks = int(cfg.get("cv", {}).get("n_movie_blocks", 5))
rows = []
status_rows = []

ready = ready_sessions(asset_index, "ready_for_latent_decoding")
print("Sessions ready for latent decoding:", ready)

for session_id in ready:
    emb_path = Path(asset_index.loc[asset_index["session_id"]].astype(str) ==
    feature_path = choose_best_feature_table(candidate_feature_files(paths,
```

```

if feature_path is None:
    status_rows.append({"session_id": session_id, "stage": "latent_decod
    continue

features = pd.read_csv(feature_path)
targets = movie_feature_targets(features, continuous_targets)
if not targets:
    status_rows.append({"session_id": session_id, "stage": "latent_decod
    continue

groups = movie_block_groups(len(features), n_movie_blocks)

print(f"\nSession {session_id}")
print("Embedding file:", emb_path)
print("Feature file:", feature_path)
print("Feature shape:", features.shape)
print("Targets:", targets)

with np.load(emb_path, allow_pickle=False) as emb:
    for name in emb.files:
        Z = emb[name]
        if Z.ndim != 2 or Z.shape[0] != len(features):
            continue

    for target in targets:
        try:
            df, pred = evaluate_regression_cv(
                Z,
                features[target].to_numpy(dtype=float),
                groups=groups,
                model_name="ridge_cv",
                n_splits=n_movie_blocks,
            )
            if df.empty:
                continue
            df = df.copy()
            df["session_id"] = session_id
            df["representation"] = name
            df["target"] = target
            df["cv_strategy"] = "contiguous_movie_block"
            df["feature_file"] = str(feature_path)
            rows.append(df)
        except Exception as exc:
            status_rows.append({
                "session_id": session_id,
                "representation": name,
                "target": target,
                "stage": "latent_decoding",
                "status": "failed",
                "reason": repr(exc),
            })

```

//

```
latent_decoding_folds = pd.concat(rows, ignore_index=True) if rows else pd.D
save_table(latent_decoding_folds, paths.publication_tables_dir / "13_latent_
save_table(pd.DataFrame(status_rows), paths.publication_tables_dir / "13_lat

if latent_decoding_folds.empty:
    print("No latent decoding folds were created.")
else:
    summary = summarize_folds(
        latent_decoding_folds,
        group_cols=["session_id", "representation", "target", "cv_strategy",
    )
    save_table(summary, paths.publication_tables_dir / "13_latent_feature_de
    display(summary.sort_values(["spearman_r_mean"], ascending=False).head(3
```

Sessions ready for latent decoding: ['500855614', '500964514', '501271265']

Session 500855614

Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-

Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

Feature shape: (900, 105)

Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv

Session 500964514

Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-

Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

Feature shape: (900, 105)

Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv

Session 501271265

Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-

Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

Feature shape: (900, 105)

Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv

	session_id	representation	target	cv_strategy
53	500964514	pca_full	spatial_frequency_centroid	contiguous_movie_block
67	501271265	isomap	spatial_frequency_centroid	contiguous_movie_block
86	501271265	umap	rms_contrast	contiguous_movie_block
51	500964514	pca_full	rms_contrast	contiguous_movie_block
90	501271265	umap	total_spectral_power	contiguous_movie_block
60	500964514	umap	spatial_frequency_centroid	contiguous_movie_block
69	501271265	isomap	total_spectral_power	contiguous_movie_block
65	501271265	isomap	rms_contrast	contiguous_movie_block
55	500964514	pca_full	total_spectral_power	contiguous_movie_block

81	501271265	pca_full	spatial_frequency_centroid	contiguous_movie_block
13	500855614	isomap	total_spectral_power	contiguous_movie_block
18	500855614	pca	spatial_frequency_centroid	contiguous_movie_block
20	500855614	pca	total_spectral_power	contiguous_movie_block
11	500855614	isomap	spatial_frequency_centroid	contiguous_movie_block
44	500964514	pca	rms_contrast	contiguous_movie_block
36	500964514	isomap	orientation_selectivity	contiguous_movie_block
29	500855614	umap	orientation_selectivity	contiguous_movie_block
54	500964514	pca_full	temporal_absdiff_rms	contiguous_movie_block
52	500964514	pca_full	scene_change_score	contiguous_movie_block
50	500964514	pca_full	orientation_selectivity	contiguous_movie_block
41	500964514	isomap	total_spectral_power	contiguous_movie_block
85	501271265	umap	orientation_selectivity	contiguous_movie_block
4	500855614	cebra	spatial_frequency_centroid	contiguous_movie_block
72	501271265	pca	rms_contrast	contiguous_movie_block
84	501271265	umap	luminance_std	contiguous_movie_block
76	501271265	pca	total_spectral_power	contiguous_movie_block
48	500964514	pca	total_spectral_power	contiguous_movie_block
25	500855614	pca_full	spatial_frequency_centroid	contiguous_movie_block
83	501271265	pca_full	total_spectral_power	contiguous_movie_block
49	500964514	pca_full	luminance_std	contiguous_movie_block

30 rows × 29 columns

```
# -----
# Common-embedding multi-session summary
# -----

common_embeddings = ["pca", "pca_full", "umap", "isomap"]

if latent_decoding_folds.empty:
    print("No latent decoding folds available.")
else:
    common_folds = latent_decoding_folds[
        latent_decoding_folds["representation"].isin(common_embeddings)
    ]
```

```

J.copy()

common_summary = summarize_folds(
    common_folds,
    group_cols=["representation", "target", "cv_strategy", "model"],
)

save_table(
    common_summary,
    paths.publication_tables_dir / "13_latent_feature_decoding_common_em
)

display(
    common_summary
    .sort_values("spearman_r_mean", ascending=False)
    .head(30)
)

```

	representation	target	cv_strategy	model	fold
18	pca_full	spatial_frequency_centroid	contiguous_movie_block	ridge_cv	
6	isomap	total_spectral_power	contiguous_movie_block	ridge_cv	
2	isomap	rms_contrast	contiguous_movie_block	ridge_cv	
4	isomap	spatial_frequency_centroid	contiguous_movie_block	ridge_cv	
13	pca	total_spectral_power	contiguous_movie_block	ridge_cv	
9	pca	rms_contrast	contiguous_movie_block	ridge_cv	
20	pca_full	total_spectral_power	contiguous_movie_block	ridge_cv	
1	isomap	orientation_selectivity	contiguous_movie_block	ridge_cv	
16	pca_full	rms_contrast	contiguous_movie_block	ridge_cv	
11	pca	spatial_frequency_centroid	contiguous_movie_block	ridge_cv	
15	pca_full	orientation_selectivity	contiguous_movie_block	ridge_cv	
23	umap	rms_contrast	contiguous_movie_block	ridge_cv	
19	pca_full	temporal_absdiff_rms	contiguous_movie_block	ridge_cv	
17	pca_full	scene_change_score	contiguous_movie_block	ridge_cv	
27	umap	total_spectral_power	contiguous_movie_block	ridge_cv	
25	umap	spatial_frequency_centroid	contiguous_movie_block	ridge_cv	
22	umap	orientation_selectivity	contiguous_movie_block	ridge_cv	
5	isomap	temporal_absdiff_rms	contiguous_movie_block	ridge_cv	
3	isomap	scene_change_score	contiguous_movie_block	ridge_cv	

21	umap	luminance_std	contiguous_movie_block	ridge_cv
12	pca	temporal_absdiff_rms	contiguous_movie_block	ridge_cv
10	pca	scene_change_score	contiguous_movie_block	ridge_cv
26	umap	temporal_absdiff_rms	contiguous_movie_block	ridge_cv
24	umap	scene_change_score	contiguous_movie_block	ridge_cv
8	pca	orientation_selectivity	contiguous_movie_block	ridge_cv
0	isomap	luminance_std	contiguous_movie_block	ridge_cv
14	pca_full	luminance_std	contiguous_movie_block	ridge_cv
7	pca	luminance_std	contiguous_movie_block	ridge_cv

28 rows × 28 columns

✓ Optional observed-vs-null controls for latent decoding

For a first test, set `RUN_EXPENSIVE_PUBLICATION_NULLS=1` and `PUBLICATION_NULL_PERMUTATIONS=100`. For final manuscript tables, use 1000+ permutations.

```
# -----
# Notebook 13: multi-session latent decoding null controls
# -----
# This runs null controls only for embeddings shared across all sessions:
# pca, pca_full, umap, isomap.
#
# Start with N_PERMUTATIONS = 100 for testing.
# Use N_PERMUTATIONS = 1000 for manuscript-grade final run.
# -----

import os
from v1_manifold_publication.decoding import evaluate_with_nulls, movie_bloc

# -----
# Settings
# -----

RUN_EXPENSIVE_NULLS = True

# For first test run, use 100.
# For final publication run, change to 1000.
N_PERMUTATIONS = 100
```

```

common_embeddings = ["pca", "pca_full", "umap", "isomap"]

print("RUN_EXPENSIVE_NULLS:", RUN_EXPENSIVE_NULLS)
print("N_PERMUTATIONS:", N_PERMUTATIONS)
print("Common embeddings:", common_embeddings)

# Use refreshed asset_index directly, not stale helper output.
ready = (
    asset_index
    .loc[asset_index["ready_for_latent_decoding"], "session_id"]
    .astype(str)
    .tolist()
)

print("Sessions ready for latent decoding:", ready)

if len(ready) < 2:
    raise RuntimeError(
        "Fewer than two sessions are ready. Multi-session null controls requ
    )

# -----
# Run null controls
# -----

if not RUN_EXPENSIVE_NULLS:
    print("Skipped expensive null controls.")
else:
    null_rows = []
    status_rows = []

    for session_id in ready:
        emb_path = Path(
            asset_index.loc[
                asset_index["session_id"].astype(str) == str(session_id),
                "embedding_file",
            ].iloc[0]
        )

        feature_path = choose_best_feature_table(
            candidate_feature_files(paths, session_id)
        )

        if feature_path is None:
            status_rows.append({
                "session_id": session_id,
                "stage": "null_controls",
                "status": "skipped",
                "reason": "No feature table found.",
            })

```

```

        continue

features = pd.read_csv(feature_path)
targets = movie_feature_targets(features, continuous_targets)

if not targets:
    status_rows.append({
        "session_id": session_id,
        "stage": "null_controls",
        "status": "skipped",
        "reason": "No target columns found.",
    })
    continue

groups = movie_block_groups(len(features), n_movie_blocks)

print("\n" + "=" * 80)
print("Session:", session_id)
print("Embedding file:", emb_path)
print("Feature file:", feature_path)
print("Feature shape:", features.shape)
print("Targets:", targets)

with np.load(emb_path, allow_pickle=False) as emb:
    available_embeddings = [
        name for name in common_embeddings
        if name in emb.files
        and emb[name].ndim == 2
        and emb[name].shape[0] == len(features)
    ]

print("Available common embeddings:", available_embeddings)

for name in available_embeddings:
    Z = emb[name]

    for target in targets:
        print(
            f"Null controls: session={session_id}, "
            f"representation={name}, target={target}"
        )

        try:
            real, nulls = evaluate_with_nulls(
                Z,
                features[target].to_numpy(dtype=float),
                groups=groups,
                task="regression",
                n_permutations=N_PERMUTATIONS,
                random_state=cfg["project"]["random_seed"],
                model_name="ridge_cv".

```

```

        model_name = stage_name,
    )

    for kind, table in [("observed", real), ("null", null_rows)]:
        if table is None or table.empty:
            continue

        table = table.copy()
        table["row_type"] = kind
        table["session_id"] = session_id
        table["representation"] = name
        table["target"] = target
        table["feature_file"] = str(feature_path)
        table["n_permutations_requested"] = N_PERMUTATIONS

        null_rows.append(table)

    status_rows.append({
        "session_id": session_id,
        "representation": name,
        "target": target,
        "stage": "null_controls",
        "status": "completed",
    })

except Exception as exc:
    status_rows.append({
        "session_id": session_id,
        "representation": name,
        "target": target,
        "stage": "null_controls",
        "status": "failed",
        "reason": repr(exc),
    })

latent_nulls = (
    pd.concat(null_rows, ignore_index=True)
    if null_rows
    else pd.DataFrame()
)

null_status = pd.DataFrame(status_rows)

save_table(
    latent_nulls,
    paths.publication_tables_dir / "13_latent_feature_decoding_null_controls"
)

save_table(
    null_status,
    paths.publication_tables_dir / "13_latent_feature_decoding_null_status"
)

```

```

)

print("\nNull-control table shape:", latent_nulls.shape)

if not latent_nulls.empty:
    display(latent_nulls.head())
    display(latent_nulls["row_type"].value_counts(dropna=False))

display(null_status)

```

```

RUN_EXPENSIVE_NULLS: True
N_PERMUTATIONS: 100
Common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
Sessions ready for latent decoding: ['500855614', '500964514', '501271265']

=====
Session: 500855614
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
Feature shape: (900, 105)
Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
Null controls: session=500855614, representation=pca, target=rms_contrast
Null controls: session=500855614, representation=pca, target=spatial_freque
Null controls: session=500855614, representation=pca, target=orientation_sele
Null controls: session=500855614, representation=pca, target=luminance_std
Null controls: session=500855614, representation=pca, target=total_spectral_p
Null controls: session=500855614, representation=pca, target=temporal_absdif
Null controls: session=500855614, representation=pca, target=scene_change_sco
Null controls: session=500855614, representation=pca_full, target=rms_contrast
Null controls: session=500855614, representation=pca_full, target=spatial_fre
Null controls: session=500855614, representation=pca_full, target=orientation
Null controls: session=500855614, representation=pca_full, target=luminance_s
Null controls: session=500855614, representation=pca_full, target=total_spect
Null controls: session=500855614, representation=pca_full, target=temporal_ab
Null controls: session=500855614, representation=pca_full, target=scene_chang
Null controls: session=500855614, representation=umap, target=rms_contrast
Null controls: session=500855614, representation=umap, target=spatial_freque
Null controls: session=500855614, representation=umap, target=orientation_sel
Null controls: session=500855614, representation=umap, target=luminance_std
Null controls: session=500855614, representation=umap, target=total_spectral_
Null controls: session=500855614, representation=umap, target=temporal_absdif
Null controls: session=500855614, representation=umap, target=scene_change_sc
Null controls: session=500855614, representation=isomap, target=rms_contrast
Null controls: session=500855614, representation=isomap, target=spatial_freque
Null controls: session=500855614, representation=isomap, target=orientation_s
Null controls: session=500855614, representation=isomap, target=luminance_std
Null controls: session=500855614, representation=isomap, target=total_spectra
Null controls: session=500855614, representation=isomap, target=temporal_absd
Null controls: session=500855614, representation=isomap, target=scene_change_

=====
Session: 500964514
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-

```

```
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
Feature shape: (900, 105)
Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
Null controls: session=500964514, representation=pca, target=rms_contrast
Null controls: session=500964514, representation=pca, target=spatial_frequenc
Null controls: session=500964514, representation=pca, target=orientation_sele
Null controls: session=500964514, representation=pca, target=luminance_std
Null controls: session=500964514, representation=pca, target=total_spectral_p
Null controls: session=500964514, representation=pca, target=temporal_absdiff
Null controls: session=500964514, representation=pca, target=scene_change_sco
Null controls: session=500964514, representation=pca_full, target=rms_contrast
Null controls: session=500964514, representation=pca_full, target=spatial_fre
Null controls: session=500964514, representation=pca_full, target=orientation
Null controls: session=500964514, representation=pca_full, target=luminance_s
Null controls: session=500964514, representation=pca_full, target=total_spect
Null controls: session=500964514, representation=pca_full, target=temporal_ab
Null controls: session=500964514, representation=pca_full, target=scene_chang
Null controls: session=500964514, representation=umap, target=rms_contrast
Null controls: session=500964514, representation=umap, target=spatial_frequen
Null controls: session=500964514, representation=umap, target=orientation_sel
Null controls: session=500964514, representation=umap, target=luminance_std
Null controls: session=500964514, representation=umap, target=total_spectral_
Null controls: session=500964514, representation=umap, target=temporal_absdif
Null controls: session=500964514, representation=umap, target=scene_change_sc
Null controls: session=500964514, representation=isomap, target=rms_contrast
Null controls: session=500964514, representation=isomap, target=spatial_frequ
Null controls: session=500964514, representation=isomap, target=orientation_s
Null controls: session=500964514, representation=isomap, target=luminance_std
Null controls: session=500964514, representation=isomap, target=total_spectra
Null controls: session=500964514, representation=isomap, target=temporal_absd
Null controls: session=500964514, representation=isomap, target=scene_change_
```

```
=====
Session: 501271265
```

```
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
Feature shape: (900, 105)
Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
Null controls: session=501271265, representation=pca, target=rms_contrast
Null controls: session=501271265, representation=pca, target=spatial_frequenc
Null controls: session=501271265, representation=pca, target=orientation_sele
Null controls: session=501271265, representation=pca, target=luminance_std
Null controls: session=501271265, representation=pca, target=total_spectral_p
Null controls: session=501271265, representation=pca, target=temporal_absdiff
Null controls: session=501271265, representation=pca, target=scene_change_sco
Null controls: session=501271265, representation=pca_full, target=rms_contrast
Null controls: session=501271265, representation=pca_full, target=spatial_fre
Null controls: session=501271265, representation=pca_full, target=orientation
Null controls: session=501271265, representation=pca_full, target=luminance_s
Null controls: session=501271265, representation=pca_full, target=total_spect
Null controls: session=501271265, representation=pca_full, target=temporal_ab
Null controls: session=501271265, representation=pca_full, target=scene_chang
```

Null controls: session=501271265, representation=umap, target=rms_contrast
Null controls: session=501271265, representation=umap, target=spatial_frequen
Null controls: session=501271265, representation=umap, target=orientation_sel
Null controls: session=501271265, representation=umap, target=luminance_std
Null controls: session=501271265, representation=umap, target=total_spectral
Null controls: session=501271265, representation=umap, target=temporal_absdif
Null controls: session=501271265, representation=umap, target=scene_change_sc
Null controls: session=501271265, representation=isomap, target=rms_contrast
Null controls: session=501271265, representation=isomap, target=spatial_frequ
Null controls: session=501271265, representation=isomap, target=orientation_s
Null controls: session=501271265, representation=isomap, target=luminance_std
Null controls: session=501271265, representation=isomap, target=total_spectra
Null controls: session=501271265, representation=isomap, target=temporal_absd
Null controls: session=501271265, representation=isomap, target=scene_change_

Null-control table shape: (25620, 17)

	fold	model	n_train	n_test	r2	mae	rmse	spearman_r	pe
0	0.0	ridge_cv	720.0	180.0	-8.851627	0.156653	0.200220	-0.142753	-
1	1.0	ridge_cv	720.0	180.0	-2.397888	0.087176	0.097605	0.647641	
2	2.0	ridge_cv	720.0	180.0	-0.214977	0.078026	0.095958	0.330372	
3	3.0	ridge_cv	720.0	180.0	-9.523626	0.274058	0.284175	0.307295	
4	4.0	ridge_cv	720.0	180.0	-1.035430	0.074584	0.100475	-0.098863	-

null 25200

observed 420

Name: row_type, dtype: int64

	session_id	representation	target	stage	status
0	500855614	pca	rms_contrast	null_controls	completed
1	500855614	pca	spatial_frequency_centroid	null_controls	completed
2	500855614	pca	orientation_selectivity	null_controls	completed
3	500855614	pca	luminance_std	null_controls	completed
4	500855614	pca	total_spectral_power	null_controls	completed
...	...	...	...	...	...
79	501271265	isomap	orientation_selectivity	null_controls	completed
80	501271265	isomap	luminance_std	null_controls	completed
81	501271265	isomap	total_spectral_power	null_controls	completed
82	501271265	isomap	temporal_absdiff_rms	null_controls	completed

```

# -----
# Notebook 13: direct manual circular-shift null controls
# -----
# This replaces the malformed null-control table.
# It computes observed decoding and circular-shift null decoding directly.
# Primary metric: held-out Spearman r.
# -----

import numpy as np
import pandas as pd
from pathlib import Path

from v1_manifold_publication.decoding import evaluate_regression_cv, movie_b

# Start with 100. For final manuscript-grade run, change to 1000.
N_PERMUTATIONS = 100

common_embeddings = ["pca", "pca_full", "umap", "isomap"]

rng = np.random.default_rng(cfg["project"]["random_seed"])

ready = (
    asset_index
    .loc[asset_index["ready_for_latent_decoding"], "session_id"]
    .astype(str)
    .tolist()
)

print("Sessions ready for null controls:", ready)
print("Common embeddings:", common_embeddings)
print("N_PERMUTATIONS:", N_PERMUTATIONS)

if len(ready) < 2:
    raise RuntimeError("Need at least two ready sessions for multi-session n

summary_rows = []
null_distribution_rows = []
status_rows = []

for session_id in ready:
    emb_path = Path(
        asset_index.loc[
            asset_index["session_id"].astype(str) == str(session_id),
            "embedding_file",
        ].iloc[0]
    )

    feature_path = choose_best_feature_table(
        candidate_feature_files(paths, session_id)
    )

```



```

)

if feature_path is None:
    status_rows.append({
        "session_id": session_id,
        "status": "skipped",
        "reason": "No feature table found.",
    })
    continue

features = pd.read_csv(feature_path)
targets = movie_feature_targets(features, continuous_targets)
groups = movie_block_groups(len(features), n_movie_blocks)

print("\n" + "=" * 90)
print("Session:", session_id)
print("Embedding file:", emb_path)
print("Feature file:", feature_path)
print("Targets:", targets)

with np.load(emb_path, allow_pickle=False) as emb:
    available_embeddings = [
        name for name in common_embeddings
        if name in emb.files
        and emb[name].ndim == 2
        and emb[name].shape[0] == len(features)
    ]

    print("Available common embeddings:", available_embeddings)

    for representation in available_embeddings:
        Z = emb[representation]

        for target in targets:
            y = features[target].to_numpy(dtype=float)

            valid = np.isfinite(y)
            if valid.sum() < 50:
                status_rows.append({
                    "session_id": session_id,
                    "representation": representation,
                    "target": target,
                    "status": "skipped",
                    "reason": "Too few finite target values.",
                })
                continue

            try:
                observed_folds, _ = evaluate_regression_cv(
                    Z,
                    y,

```

```

        groups=groups,
        model_name="ridge_cv",
        n_splits=n_movie_blocks,
    )

    if observed_folds.empty or "spearman_r" not in observed_
        status_rows.append({
            "session_id": session_id,
            "representation": representation,
            "target": target,
            "status": "skipped",
            "reason": "Observed decoding returned no usable
        })
        continue

    observed_mean = float(observed_folds["spearman_r"].mean(
    observed_median = float(observed_folds["spearman_r"].med
    observed_std = float(observed_folds["spearman_r"].std())
    observed_r2_mean = (
        float(observed_folds["r2"].mean())
        if "r2" in observed_folds.columns
        else np.nan
    )

    null_values = []

    n = len(y)
    min_shift = max(10, n // 10)
    max_shift = n - min_shift

    if max_shift <= min_shift:
        possible_shifts = np.arange(1, n)
    else:
        possible_shifts = np.arange(min_shift, max_shift)

    print(
        f"Nulls: session={session_id}, representation={repre
        f"target={target}, observed_spearman={observed_mean:
    )

    for perm in range(N_PERMUTATIONS):
        shift = int(rng.choice(possible_shifts))
        y_null = np.roll(y, shift)

        null_folds, _ = evaluate_regression_cv(
            Z,
            y_null,
            groups=groups,
            model_name="ridge_cv",
            n_splits=n_movie_blocks,

```

```

)

if null_folds.empty or "spearman_r" not in null_fold
    null_mean = np.nan
    null_median = np.nan
else:
    null_mean = float(null_folds["spearman_r"].mean(
    null_median = float(null_folds["spearman_r"].med

null_values.append(null_mean)

null_distribution_rows.append({
    "session_id": session_id,
    "representation": representation,
    "target": target,
    "permutation": perm,
    "shift": shift,
    "null_mean_spearman": null_mean,
    "null_median_spearman": null_median,
})

null_values = np.asarray(null_values, dtype=float)
null_values = null_values[np.isfinite(null_values)]

if len(null_values) == 0:
    empirical_p = np.nan
    null_mean = np.nan
    null_median = np.nan
    null_std = np.nan
    effect_over_null = np.nan
    null_z = np.nan
else:
    null_mean = float(np.mean(null_values))
    null_median = float(np.median(null_values))
    null_std = float(np.std(null_values, ddof=0))
    effect_over_null = observed_mean - null_mean
    null_z = effect_over_null / (null_std + 1e-12)

    empirical_p = (
        1 + np.sum(null_values >= observed_mean)
    ) / (len(null_values) + 1)

summary_rows.append({
    "session_id": session_id,
    "representation": representation,
    "target": target,
    "null_type": "circular_shift_target",
    "observed_mean_spearman": observed_mean,
    "observed_median_spearman": observed_median,
    "observed_std_spearman": observed_std,
    "observed_mean_r2": observed_r2_mean,

```

```

        "n_observed_folds": int(len(observed_folds)),
        "n_null_permutations": int(len(null_values)),
        "null_mean_spearman": null_mean,
        "null_median_spearman": null_median,
        "null_std_spearman": null_std,
        "effect_over_null_spearman": effect_over_null,
        "null_z_score": null_z,
        "empirical_p_one_sided": empirical_p,
    })

    status_rows.append({
        "session_id": session_id,
        "representation": representation,
        "target": target,
        "status": "completed",
        "n_null_permutations": int(len(null_values)),
    })

except Exception as exc:
    status_rows.append({
        "session_id": session_id,
        "representation": representation,
        "target": target,
        "status": "failed",
        "reason": repr(exc),
    })

null_summary = pd.DataFrame(summary_rows)
null_distribution = pd.DataFrame(null_distribution_rows)
null_status = pd.DataFrame(status_rows)

# -----
# Benjamini-Hochberg FDR correction
# -----

def benjamini_hochberg(p_values):
    p = np.asarray(p_values, dtype=float)
    q = np.full_like(p, np.nan, dtype=float)

    valid = np.isfinite(p)
    if valid.sum() == 0:
        return q

    p_valid = p[valid]
    n = len(p_valid)

    order = np.argsort(p_valid)
    ranked_p = p_valid[order]

```

```

ranked_q = ranked_p * n / (np.arange(1, n + 1))
ranked_q = np.minimum.accumulate(ranked_q[::-1])[::-1]
ranked_q = np.clip(ranked_q, 0, 1)

q_valid = np.empty_like(p_valid)
q_valid[order] = ranked_q
q[valid] = q_valid

return q

if not null_summary.empty:
    null_summary["empirical_q_bh"] = benjamini_hochberg(
        null_summary["empirical_p_one_sided"].to_numpy()
    )
    null_summary["significant_fdr_0_05"] = (
        null_summary["empirical_q_bh"] <= 0.05
    )

save_table(
    null_summary,
    paths.publication_tables_dir / "13_latent_feature_decoding_null_summary.
)

save_table(
    null_distribution,
    paths.publication_tables_dir / "13_latent_feature_decoding_null_distribu
)

save_table(
    null_status,
    paths.publication_tables_dir / "13_latent_feature_decoding_null_status.c
)

print("\nNull summary shape:", null_summary.shape)
print("Null distribution shape:", null_distribution.shape)

if not null_summary.empty:
    print("Minimum n_null_permutations:", null_summary["n_null_permutations"]
    print("Maximum n_null_permutations:", null_summary["n_null_permutations"]
    print("Number FDR-significant:", int(null_summary["significant_fdr_0_05"]

    display(
        null_summary
        .sort_values(
            ["empirical_q_bh", "observed_mean_spearman"],
            ascending=[True, False],
        )
        .head(40)
    )
)

```

display(null_status)

```
Sessions ready for null controls: ['500855614', '500964514', '501271265']
Common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
N_PERMUTATIONS: 100

=====
Session: 500855614
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
Nulls: session=500855614, representation=pca, target=rms_contrast, observed_s
Nulls: session=500855614, representation=pca, target=spatial_frequency_centro
Nulls: session=500855614, representation=pca, target=orientation_selectivity,
Nulls: session=500855614, representation=pca, target=luminance_std, observed_
Nulls: session=500855614, representation=pca, target=total_spectral_power, ob
Nulls: session=500855614, representation=pca, target=temporal_absdiff_rms, ob
Nulls: session=500855614, representation=pca, target=scene_change_score, obse
Nulls: session=500855614, representation=pca_full, target=rms_contrast, obser
Nulls: session=500855614, representation=pca_full, target=spatial_frequency_c
Nulls: session=500855614, representation=pca_full, target=orientation_selecti
Nulls: session=500855614, representation=pca_full, target=luminance_std, obse
Nulls: session=500855614, representation=pca_full, target=total_spectral_powe
Nulls: session=500855614, representation=pca_full, target=temporal_absdiff_rm
Nulls: session=500855614, representation=pca_full, target=scene_change_score,
Nulls: session=500855614, representation=umap, target=rms_contrast, observed_
Nulls: session=500855614, representation=umap, target=spatial_frequency_cent
Nulls: session=500855614, representation=umap, target=orientation_selectivity
Nulls: session=500855614, representation=umap, target=luminance_std, observed
Nulls: session=500855614, representation=umap, target=total_spectral_power, o
Nulls: session=500855614, representation=umap, target=temporal_absdiff_rms, o
Nulls: session=500855614, representation=umap, target=scene_change_score, obs
Nulls: session=500855614, representation=isomap, target=rms_contrast, observe
Nulls: session=500855614, representation=isomap, target=spatial_frequency_cen
Nulls: session=500855614, representation=isomap, target=orientation_selectivi
Nulls: session=500855614, representation=isomap, target=luminance_std, observ
Nulls: session=500855614, representation=isomap, target=total_spectral_power,
Nulls: session=500855614, representation=isomap, target=temporal_absdiff_rms,
Nulls: session=500855614, representation=isomap, target=scene_change_score, o

=====
Session: 500964514
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectiv
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']
Nulls: session=500964514, representation=pca, target=rms_contrast, observed_s
Nulls: session=500964514, representation=pca, target=spatial_frequency_centro
Nulls: session=500964514, representation=pca, target=orientation_selectivity,
Nulls: session=500964514, representation=pca, target=luminance_std, observed_
Nulls: session=500964514, representation=pca, target=total_spectral_power, ob
Nulls: session=500964514, representation=pca, target=temporal_absdiff_rms, ob
```

Nulls: session=500964514, representation=pca, target=scene_change_score, observed_...
Nulls: session=500964514, representation=pca_full, target=rms_contrast, observed_...
Nulls: session=500964514, representation=pca_full, target=spatial_frequency_centroid, observed_...
Nulls: session=500964514, representation=pca_full, target=orientation_selectivity, observed_...
Nulls: session=500964514, representation=pca_full, target=luminance_std, observed_...
Nulls: session=500964514, representation=pca_full, target=total_spectral_power, observed_...
Nulls: session=500964514, representation=pca_full, target=temporal_absdiff_rms, observed_...
Nulls: session=500964514, representation=pca_full, target=scene_change_score, observed_...
Nulls: session=500964514, representation=umap, target=rms_contrast, observed_...
Nulls: session=500964514, representation=umap, target=spatial_frequency_centroid, observed_...
Nulls: session=500964514, representation=umap, target=orientation_selectivity, observed_...
Nulls: session=500964514, representation=umap, target=luminance_std, observed_...
Nulls: session=500964514, representation=umap, target=total_spectral_power, observed_...
Nulls: session=500964514, representation=umap, target=temporal_absdiff_rms, observed_...
Nulls: session=500964514, representation=umap, target=scene_change_score, observed_...
Nulls: session=500964514, representation=isomap, target=rms_contrast, observed_...
Nulls: session=500964514, representation=isomap, target=spatial_frequency_centroid, observed_...
Nulls: session=500964514, representation=isomap, target=orientation_selectivity, observed_...
Nulls: session=500964514, representation=isomap, target=luminance_std, observed_...
Nulls: session=500964514, representation=isomap, target=total_spectral_power, observed_...
Nulls: session=500964514, representation=isomap, target=temporal_absdiff_rms, observed_...
Nulls: session=500964514, representation=isomap, target=scene_change_score, observed_...

=====
Session: 501271265

Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

Targets: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_selectivity', 'luminance_std', 'total_spectral_power', 'temporal_absdiff_rms', 'scene_change_score']

Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']

Nulls: session=501271265, representation=pca, target=rms_contrast, observed_...
Nulls: session=501271265, representation=pca, target=spatial_frequency_centroid, observed_...
Nulls: session=501271265, representation=pca, target=orientation_selectivity, observed_...
Nulls: session=501271265, representation=pca, target=luminance_std, observed_...
Nulls: session=501271265, representation=pca, target=total_spectral_power, observed_...
Nulls: session=501271265, representation=pca, target=temporal_absdiff_rms, observed_...
Nulls: session=501271265, representation=pca, target=scene_change_score, observed_...
Nulls: session=501271265, representation=pca_full, target=rms_contrast, observed_...
Nulls: session=501271265, representation=pca_full, target=spatial_frequency_centroid, observed_...
Nulls: session=501271265, representation=pca_full, target=orientation_selectivity, observed_...
Nulls: session=501271265, representation=pca_full, target=luminance_std, observed_...
Nulls: session=501271265, representation=pca_full, target=total_spectral_power, observed_...
Nulls: session=501271265, representation=pca_full, target=temporal_absdiff_rms, observed_...
Nulls: session=501271265, representation=pca_full, target=scene_change_score, observed_...
Nulls: session=501271265, representation=umap, target=rms_contrast, observed_...
Nulls: session=501271265, representation=umap, target=spatial_frequency_centroid, observed_...
Nulls: session=501271265, representation=umap, target=orientation_selectivity, observed_...
Nulls: session=501271265, representation=umap, target=luminance_std, observed_...
Nulls: session=501271265, representation=umap, target=total_spectral_power, observed_...
Nulls: session=501271265, representation=umap, target=temporal_absdiff_rms, observed_...
Nulls: session=501271265, representation=umap, target=scene_change_score, observed_...
Nulls: session=501271265, representation=isomap, target=rms_contrast, observed_...
Nulls: session=501271265, representation=isomap, target=spatial_frequency_centroid, observed_...
Nulls: session=501271265, representation=isomap, target=orientation_selectivity, observed_...
Nulls: session=501271265, representation=isomap, target=luminance_std, observed_...

Nulls: session=501271265, representation=isomap, target=total_spectral_power,
Nulls: session=501271265, representation=isomap, target=temporal_absdiff_rms,
Nulls: session=501271265, representation=isomap, target=scene_change_score, o

Null summary shape: (84, 18)

Null distribution shape: (8400, 7)

Minimum n_null_permutations: 100

Maximum n_null_permutations: 100

Number FDR-significant: 0

	session_id	representation	target	null_type	obs
35	500964514	pca_full	rms_contrast	circular_shift_target	
81	501271265	isomap	total_spectral_power	circular_shift_target	
28	500964514	pca	rms_contrast	circular_shift_target	
32	500964514	pca	total_spectral_power	circular_shift_target	
47	500964514	umap	temporal_absdiff_rms	circular_shift_target	
48	500964514	umap	scene_change_score	circular_shift_target	
68	501271265	pca_full	temporal_absdiff_rms	circular_shift_target	
41	500964514	pca_full	scene_change_score	circular_shift_target	
53	500964514	isomap	total_spectral_power	circular_shift_target	
83	501271265	isomap	scene_change_score	circular_shift_target	
40	500964514	pca_full	temporal_absdiff_rms	circular_shift_target	
82	501271265	isomap	temporal_absdiff_rms	circular_shift_target	
4	500855614	pca	total_spectral_power	circular_shift_target	
51	500964514	isomap	orientation_selectivity	circular_shift_target	
36	500964514	pca_full	spatial_frequency_centroid	circular_shift_target	
69	501271265	pca_full	scene_change_score	circular_shift_target	
5	500855614	pca	temporal_absdiff_rms	circular_shift_target	
64	501271265	pca_full	spatial_frequency_centroid	circular_shift_target	
49	500964514	isomap	rms_contrast	circular_shift_target	
6	500855614	pca	scene_change_score	circular_shift_target	
77	501271265	isomap	rms_contrast	circular_shift_target	
8	500855614	pca_full	spatial_frequency_centroid	circular_shift_target	
1	500855614	pca	spatial_frequency_centroid	circular_shift_target	
56	501271265	pca	rms_contrast	circular_shift_target	
25	500855614	isomap	total spectral power	circular shift target	

39	500964514	pca_full	total_spectral_power	circular_shift_target
73	501271265	umap	luminance_std	circular_shift_target
78	501271265	isomap	spatial_frequency_centroid	circular_shift_target
72	501271265	umap	orientation_selectivity	circular_shift_target
0	500855614	pca	rms_contrast	circular_shift_target
43	500964514	umap	spatial_frequency_centroid	circular_shift_target
38	500964514	pca_full	luminance_std	circular_shift_target
31	500964514	pca	luminance_std	circular_shift_target
60	501271265	pca	total_spectral_power	circular_shift_target
30	500964514	pca	orientation_selectivity	circular_shift_target
27	500855614	isomap	scene_change_score	circular_shift_target
70	501271265	umap	rms_contrast	circular_shift_target
33	500964514	pca	temporal_absdiff_rms	circular_shift_target
26	500855614	isomap	temporal_absdiff_rms	circular_shift_target
34	500964514	pca	scene_change_score	circular_shift_target
session_id representation target status n_null_perm				
0	500855614	pca	rms_contrast	completed
1	500855614	pca	spatial_frequency_centroid	completed
2	500855614	pca	orientation_selectivity	completed
3	500855614	pca	luminance_std	completed
4	500855614	pca	total_spectral_power	completed
...	...	...	...	...
79	501271265	isomap	orientation_selectivity	completed
80	501271265	isomap	luminance_std	completed
81	501271265	isomap	total_spectral_power	completed
82	501271265	isomap	temporal_absdiff_rms	completed
83	501271265	isomap	scene_change_score	completed
84 rows × 5 columns				

```

# -----
# Multi-session null-summary aggregation across sessions
# -----

common_embeddings = ["pca", "pca_full", "umap", "isomap"]

common_null_summary = null_summary[
    null_summary["representation"].isin(common_embeddings)
].copy()

if common_null_summary.empty:
    raise RuntimeError("common_null_summary is empty. Check null_summary and

multisession_null_summary = (
    common_null_summary
    .groupby(["representation", "target"], as_index=False)
    .agg(
        n_sessions=("session_id", "nunique"),
        mean_observed_spearman=("observed_mean_spearman", "mean"),
        median_observed_spearman=("observed_mean_spearman", "median"),
        std_observed_spearman=("observed_mean_spearman", "std"),
        mean_null_spearman=("null_mean_spearman", "mean"),
        median_null_spearman=("null_mean_spearman", "median"),
        mean_effect_over_null=("effect_over_null_spearman", "mean"),
        median_effect_over_null=("effect_over_null_spearman", "median"),
        mean_null_z_score=("null_z_score", "mean"),
        median_null_z_score=("null_z_score", "median"),
        min_empirical_p=("empirical_p_one_sided", "min"),
        median_empirical_p=("empirical_p_one_sided", "median"),
        min_empirical_q=("empirical_q_bh", "min"),
        median_empirical_q=("empirical_q_bh", "median"),
        n_fdr_significant=("significant_fdr_0_05", "sum"),
    )
)

multisession_null_summary = multisession_null_summary.sort_values(
    ["n_fdr_significant", "mean_effect_over_null", "mean_observed_spearman"]
    ascending=[False, False, False],
)

save_table(
    multisession_null_summary,
    paths.publication_tables_dir / "13_latent_feature_decoding_multisession_
)

display(multisession_null_summary.head(40))

print("Saved multi-session null summary:")
print(paths.publication_tables_dir / "13_latent_feature_decoding_multisessio

```

	representation		target	n_sessions	mean_observed_spearma
6	isomap		total_spectral_power	3	0.36421
18	pca_full		spatial_frequency_centroid	3	0.38710
13	pca		total_spectral_power	3	0.29197
9	pca		rms_contrast	3	0.27143
2	isomap		rms_contrast	3	0.29887
17	pca_full		scene_change_score	3	0.16676
19	pca_full		temporal_absdiff_rms	3	0.16676
11	pca		spatial_frequency_centroid	3	0.19425
5	isomap		temporal_absdiff_rms	3	0.13771
10	pca		scene_change_score	3	0.09897
12	pca		temporal_absdiff_rms	3	0.09897
4	isomap		spatial_frequency_centroid	3	0.29751
3	isomap		scene_change_score	3	0.13771
16	pca_full		rms_contrast	3	0.22001
1	isomap		orientation_selectivity	3	0.22952
20	pca_full		total_spectral_power	3	0.26153
24	umap		scene_change_score	3	0.08886
26	umap		temporal_absdiff_rms	3	0.08886
15	pca_full		orientation_selectivity	3	0.18357
8	pca		orientation_selectivity	3	0.06422
7	pca		luminance_std	3	0.01974
21	umap		luminance_std	3	0.13611
23	umap		rms_contrast	3	0.17762
14	pca_full		luminance_std	3	0.02390
22	umap		orientation_selectivity	3	0.14344
0	isomap		luminance_std	3	0.03440
27	umap		total_spectral_power	3	0.15664
25	umap		spatial_frequency_centroid	3	0.14475
Saved multi-session null summary:					
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\r					

✓ Summarize latent decoding null controls

```
manual_summary_path = paths.publication_tables_dir / "13_latent_feature_deco
manual_distribution_path = paths.publication_tables_dir / "13_latent_feature

if manual_summary_path.exists() and manual_distribution_path.exists():
    null_summary = pd.read_csv(manual_summary_path)

    print("Using valid manual circular-shift null summary:")
    print(manual_summary_path)
    print("Shape:", null_summary.shape)

    display(
        null_summary
        .sort_values(
            ["empirical_q_bh", "observed_mean_spearman"],
            ascending=[True, False],
        )
        .head(40)
    )
else:
    print("Manual null-summary files were not found. Run the manual circular
```

Using valid manual circular-shift null summary:

C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\m
Shape: (84, 18)

	session_id	representation	target	null_type	obs
35	500964514	pca_full	rms_contrast	circular_shift_target	
81	501271265	isomap	total_spectral_power	circular_shift_target	
28	500964514	pca	rms_contrast	circular_shift_target	
32	500964514	pca	total_spectral_power	circular_shift_target	
47	500964514	umap	temporal_absdiff_rms	circular_shift_target	
48	500964514	umap	scene_change_score	circular_shift_target	
68	501271265	pca_full	temporal_absdiff_rms	circular_shift_target	
41	500964514	pca_full	scene_change_score	circular_shift_target	
53	500964514	isomap	total_spectral_power	circular_shift_target	
83	501271265	isomap	scene_change_score	circular_shift_target	
40	500964514	pca_full	temporal_absdiff_rms	circular_shift_target	
82	501271265	isomap	temporal_absdiff_rms	circular_shift_target	

4	500855614	pca	total_spectral_power	circular_shift_target
51	500964514	isomap	orientation_selectivity	circular_shift_target
36	500964514	pca_full	spatial_frequency_centroid	circular_shift_target
69	501271265	pca_full	scene_change_score	circular_shift_target
5	500855614	pca	temporal_absdiff_rms	circular_shift_target
64	501271265	pca_full	spatial_frequency_centroid	circular_shift_target
49	500964514	isomap	rms_contrast	circular_shift_target
6	500855614	pca	scene_change_score	circular_shift_target
77	501271265	isomap	rms_contrast	circular_shift_target
8	500855614	pca_full	spatial_frequency_centroid	circular_shift_target
1	500855614	pca	spatial_frequency_centroid	circular_shift_target
56	501271265	pca	rms_contrast	circular_shift_target
25	500855614	isomap	total_spectral_power	circular_shift_target
39	500964514	pca_full	total_spectral_power	circular_shift_target
73	501271265	umap	luminance_std	circular_shift_target
78	501271265	isomap	spatial_frequency_centroid	circular_shift_target
72	501271265	umap	orientation_selectivity	circular_shift_target
0	500855614	pca	rms_contrast	circular_shift_target
43	500964514	umap	spatial_frequency_centroid	circular_shift_target
38	500964514	pca_full	luminance_std	circular_shift_target
31	500964514	pca	luminance_std	circular_shift_target
60	501271265	pca	total_spectral_power	circular_shift_target
30	500964514	pca	orientation_selectivity	circular_shift_target
27	500855614	isomap	scene_change_score	circular_shift_target
70	501271265	umap	rms_contrast	circular_shift_target
33	500964514	pca	temporal_absdiff_rms	circular_shift_target
26	500855614	isomap	temporal_absdiff_rms	circular_shift_target
34	500964514	pca	scene_change_score	circular_shift_target

✓ Stimulus features → V1 population encoding

This uses all sessions with neural tensors and enhanced feature tables.

```
from v1_manifold.preprocessing import load_trial_tensor_h5
from v1_manifold_publication.encoding import ridge_population_encoding, feat
from v1_manifold_publication.decoding import movie_block_groups
from v1_manifold_publication.metrics import summarize_folds

feature_cols = [
    "rms_contrast",
    "spatial_frequency_centroid",
    "orientation_selectivity",
    "luminance_std",
    "total_spectral_power",
    "temporal_absdiff_rms",
    "scene_change_score",
]

n_movie_blocks = int(cfg.get("cv", {}).get("n_movie_blocks", 5))
ridge_alphas = cfg.get("models", {}).get("ridge_alphas", [0.1, 1.0, 10.0, 10

encoding_rows = []
status_rows = []

ready = (
    asset_index
    .loc[asset_index["ready_for_population_encoding"], "session_id"]
    .astype(str)
    .tolist()
)

print("Sessions ready for population encoding:", ready)

for session_id in ready:
    tensor_path = Path(
        asset_index.loc[
            asset_index["session_id"].astype(str) == str(session_id),
            "tensor_file",
        ].iloc[0]
    )

    feature_path = choose_best_feature_table(
        candidate_feature_files(paths, session_id)
    )

    try:
        tensor, R = load_trial_tensor_h5(tensor_path)
        features = pd.read_csv(feature_path)
```

```

cols = movie_feature_targets(features, feature_cols)

if not cols:
    raise ValueError("No requested feature columns found.")

if len(features) != R.shape[0]:
    raise ValueError(
        f"Feature/R length mismatch: features={len(features)}, R={R.
    )

X = feature_matrix_from_table(features, cols)
groups = movie_block_groups(len(features), n_movie_blocks)

print(f"\nSession {session_id}")
print("Feature file:", feature_path)
print("Tensor file:", tensor_path)
print("X shape:", X.shape)
print("R shape:", R.shape)
print("Tensor shape:", tensor.shape)
print("Feature columns:", cols)

df, pred = ridge_population_encoding(
    X,
    R,
    groups=groups,
    n_splits=n_movie_blocks,
    alphas=ridge_alphas,
)

if not df.empty:
    df = df.copy()
    df["session_id"] = session_id
    df["feature_set"] = "+".join(cols)
    df["tensor_file"] = str(tensor_path)
    df["feature_file"] = str(feature_path)
    encoding_rows.append(df)

    status_rows.append({
        "session_id": session_id,
        "stage": "population_encoding",
        "status": "completed",
        "n_rows": len(df),
    })

except Exception as exc:
    status_rows.append({
        "session_id": session_id,
        "stage": "population_encoding",
        "status": "failed",
        "reason": repr(exc),
    })

```

```

    encoding = (
        pd.concat(encoding_rows, ignore_index=True)
        if encoding_rows
        else pd.DataFrame()
    )

    encoding_status = pd.DataFrame(status_rows)

    save_table(
        encoding,
        paths.publication_tables_dir / "13_stimulus_population_encoding_folds.cs
    )

    save_table(
        encoding,
        paths.publication_tables_dir / "13_stimulus_to_population_encoding_folds
    )

    save_table(
        encoding_status,
        paths.publication_tables_dir / "13_stimulus_population_encoding_status.c
    )

    if encoding.empty:
        print("No population-encoding rows were produced.")
        display(encoding_status)
    else:
        group_cols = (
            ["session_id", "feature_set", "model"]
            if "model" in encoding.columns
            else ["session_id", "feature_set"]
        )

        encoding_summary = summarize_folds(
            encoding,
            group_cols=group_cols,
        )

        save_table(
            encoding_summary,
            paths.publication_tables_dir / "13_stimulus_population_encoding_summ
        )

        save_table(
            encoding_summary,
            paths.publication_tables_dir / "13_stimulus_to_population_encoding_s
        )

        display(encoding_summary.head(30))

```



```
display(encoding_status)
```

```
Sessions ready for population encoding: ['500855614', '500964514', '501271265']
```

```
Session 500855614
```

```
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

```
Tensor file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

```
X shape: (900, 7)
```

```
R shape: (900, 163)
```

```
Tensor shape: (10, 163, 900)
```

```
Feature columns: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_
```

```
Session 500964514
```

```
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

```
Tensor file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

```
X shape: (900, 7)
```

```
R shape: (900, 214)
```

```
Tensor shape: (10, 214, 900)
```

```
Feature columns: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_
```

```
Session 501271265
```

```
Feature file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

```
Tensor file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
```

```
X shape: (900, 7)
```

```
R shape: (900, 215)
```

```
Tensor shape: (10, 215, 900)
```

```
Feature columns: ['rms_contrast', 'spatial_frequency_centroid', 'orientation_
```

	session_id	feature_set	fold_mean	fold_std
0	500855614	rms_contrast+spatial_frequency_centroid+orient...	2.0	1.581139
1	500964514	rms_contrast+spatial_frequency_centroid+orient...	2.0	1.581139
2	501271265	rms_contrast+spatial_frequency_centroid+orient...	2.0	1.581139

```
3 rows x 35 columns
```

	session_id	stage	status	n_rows
0	500855614	population_encoding	completed	5
1	500964514	population_encoding	completed	5
2	501271265	population_encoding	completed	5

✓ Hierarchy-based population encoding

This compares luminance/contrast, spatial/orientation, temporal/motion, V1-like Gabor, and optional DNN feature groups.

```

# -----
# Notebook 13: hierarchy-based population encoding comparison
# -----

from v1_manifold.preprocessing import load_trial_tensor_h5
from v1_manifold_publication.stimulus_features import feature_hierarchy_colu
from v1_manifold_publication.encoding import (
    evaluate_population_encoding_cv,
    reduced_rank_population_encoding,
    feature_matrix_from_table,
)
from v1_manifold_publication.decoding import movie_block_groups
from v1_manifold_publication.metrics import summarize_folds

hierarchy_rows = []
status_rows = []

n_movie_blocks = int(cfg.get("cv", {}).get("n_movie_blocks", 5))
ridge_alphas = cfg.get("models", {}).get("ridge_alphas", [0.1, 1.0, 10.0, 10

primary_models = cfg.get(
    "encoding_models",
    {}
).get(
    "primary_models",
    ["ridge_cv", "elastic_net", "reduced_rank_ridge"]
)

ready = (
    asset_index
    .loc[asset_index["ready_for_population_encoding"], "session_id"]
    .astype(str)
    .tolist()
)

print("Sessions ready for hierarchy encoding:", ready)
print("Primary models:", primary_models)

for session_id in ready:
    tensor_path = Path(
        asset_index.loc[
            asset_index["session_id"].astype(str) == str(session_id),
            "tensor_file",
        ].iloc[0]
    )

    feature_path = choose_best_feature_table(
        candidate_feature_files(paths, session_id)
    )

    if feature_path is None:

```

```

        status_rows.append({
            "session_id": session_id,
            "stage": "feature_hierarchy_encoding",
            "status": "skipped",
            "reason": "No feature table found.",
        })
        continue

    try:
        tensor, R = load_trial_tensor_h5(tensor_path)
        features = pd.read_csv(feature_path)

        if len(features) != R.shape[0]:
            raise ValueError(
                f"Feature/R length mismatch: features={len(features)}, R={R.
            )

        groups = movie_block_groups(len(features), n_movie_blocks)
        hierarchy = feature_hierarchy_columns(features)

        print("\n" + "=" * 90)
        print("Session:", session_id)
        print("Tensor:", tensor_path)
        print("Features:", feature_path)
        print("R shape:", R.shape)
        print("Available hierarchy groups:")
        for group, cols in hierarchy.items():
            print(f" {group}: {len(cols)} features")

        for feature_group, cols in hierarchy.items():
            if not cols:
                status_rows.append({
                    "session_id": session_id,
                    "feature_group": feature_group,
                    "stage": "feature_hierarchy_encoding",
                    "status": "skipped",
                    "reason": "No columns available for this feature group."
                })
                continue

        X = feature_matrix_from_table(features, cols)

        for model_name in primary_models:
            try:
                if model_name == "reduced_rank_ridge":
                    df, pred = reduced_rank_population_encoding(
                        X,
                        R,
                        groups=groups,
                        n_splits=n_movie_blocks,
                        alpha=alpha,

```

```

        alphas=ridge_alphas,
    )
else:
    df, pred = evaluate_population_encoding_cv(
        X,
        R,
        groups=groups,
        model_name=model_name,
        n_splits=n_movie_blocks,
        alphas=ridge_alphas,
    )

if df is None or df.empty:
    status_rows.append({
        "session_id": session_id,
        "feature_group": feature_group,
        "model": model_name,
        "stage": "feature_hierarchy_encoding",
        "status": "empty_result",
    })
    continue

df = df.copy()
df["session_id"] = session_id
df["feature_group"] = feature_group
df["n_stimulus_features"] = len(cols)
df["feature_file"] = str(feature_path)
df["tensor_file"] = str(tensor_path)

hierarchy_rows.append(df)

status_rows.append({
    "session_id": session_id,
    "feature_group": feature_group,
    "model": model_name,
    "stage": "feature_hierarchy_encoding",
    "status": "completed",
    "n_rows": len(df),
})

except Exception as exc:
    status_rows.append({
        "session_id": session_id,
        "feature_group": feature_group,
        "model": model_name,
        "stage": "feature_hierarchy_encoding",
        "status": "failed",
        "reason": repr(exc),
    })

except Exception as exc:

```

```

        status_rows.append({
            "session_id": session_id,
            "stage": "feature_hierarchy_encoding",
            "status": "failed",
            "reason": repr(exc),
        })

hierarchy_encoding = (
    pd.concat(hierarchy_rows, ignore_index=True)
    if hierarchy_rows
    else pd.DataFrame()
)

hierarchy_status = pd.DataFrame(status_rows)

save_table(
    hierarchy_encoding,
    paths.publication_tables_dir / "13_feature_hierarchy_encoding_folds.csv"
)

save_table(
    hierarchy_status,
    paths.publication_tables_dir / "13_feature_hierarchy_encoding_status.csv"
)

if hierarchy_encoding.empty:
    print("No hierarchy-encoding rows were produced.")
    display(hierarchy_status)
else:
    metric_col = (
        "population_spearman_r"
        if "population_spearman_r" in hierarchy_encoding.columns
        else "population_pearson_r"
        if "population_pearson_r" in hierarchy_encoding.columns
        else "population_r2"
    )

    hierarchy_encoding_summary = summarize_folds(
        hierarchy_encoding.dropna(subset=[metric_col]),
        group_cols=["session_id", "feature_group", "model"],
    )

    save_table(
        hierarchy_encoding_summary,
        paths.publication_tables_dir / "13_feature_hierarchy_encoding_summar
    )

    display(
        hierarchy_encoding_summary
        .sort_values(
            ["{metric_col} mean"] if "{metric_col} mean" in hierarchy_enco

```

```
        ['metric1_c01f_mean'] + ['metric1_c01f_mean'] + hierarchy_endc
        ascending=False,
    )
    .head(40)
)

display(hierarchy_status)
```


